

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

Exposición 1er Corte

TEMA:

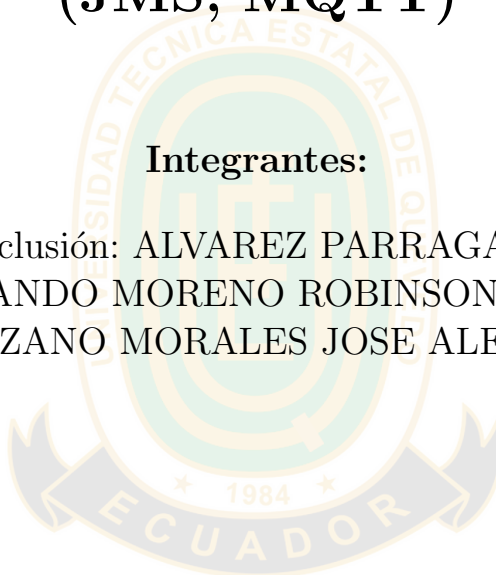
Mensajería en Aplicaciones distribuidas (JMS, MQTT)

Integrantes:

Introducción y Conclusión: ALVAREZ PARRAGA JEREMY ALEXIS

MQTT: CANDO MORENO ROBINSON RODRIGO

JMS: LOZANO MORALES JOSE ALEJANDRO



Docente Guía:

ING. GUERRERO ULLOA GLEISTON CICERON

Quevedo — Ecuador

2026

Índice

1. Introducción	2
1.1. Tipos de Mensajería y Patrones Distribuidos	2
2. Análisis Técnico Integral del Protocolo MQTT	2
2.1. Niveles de Calidad de Servicio (QoS)	3
2.2. Vectores de Ciberseguridad y Resiliencia del Broker	3
3. Experimentación y Arquitectura del Estándar JMS	4
3.1. Comportamiento Empírico de la Máquina Virtual de Java (JVM)	5
3.2. Casos de Aplicación Industrial y de Alta Seguridad	5
4. Conclusión	6

1. Introducción

La mensajería es un mecanismo central para integrar servicios en aplicaciones distribuidas modernas, permitiendo comunicación asíncrona, desacoplamiento y escalabilidad. Las arquitecturas contemporáneas se fundamentan en el intercambio de información a través del paso de mensajes como alternativa matemática e ingenieril a los entornos de memoria compartida tradicional [1]. Este enfoque metodológico permite modelar interacciones complejas mediante canales orientados al envío síncrono y asíncrono, abstrayendo construcciones críticas de concurrencia como búferes acotados y esquemas cliente-servidor dinámicos.

En el espectro de los sistemas distribuidos, coexisten diversos paradigmas que abarcan desde el acoplamiento rígido del *rendez-vous* síncrono hasta comunicaciones asíncronas caracterizadas por comportamientos concurrentes altamente desordenados. Con el propósito de formalizar estas interacciones, la literatura científica postula una jerarquía unificada de modelos de paso de mensajes cimentada estrictamente en las garantías del orden de entrega, tales como las propiedades FIFO, ordenamiento causal y orden total [2].

En la práctica computacional corporativa, la comunicación entre procesos (IPC) se bifurca metodológicamente entre el acceso a memoria compartida y el intercambio formal de mensajes, el cual abarca tuberías (*pipes*), sockets Berkeley, colas de mensajes estructuradas y el patrón publicador/suscriptor (*Publish/Subscribe*) [3], [4].

1.1. Tipos de Mensajería y Patrones Distribuidos

El diseño de infraestructuras asíncronas requiere caracterizar con precisión los patrones de transferencia de datos. Las colas de mensajes estructuran una comunicación asíncrona robusta donde el emisor y el receptor se encuentran completamente desacoplados en tiempo y espacio; el mensaje se deposita en un contenedor lógico persistente hasta que el consumidor se halle disponible para procesarlo [5].

Por otro lado, el modelo de Productor/Consumidor distribuye la carga operativa de manera eficiente a través de múltiples nodos de trabajo, garantizando que cada elemento de datos sea procesado por una única entidad activa. En contraste, el patrón Publicador/Suscriptor expande estas capacidades hacia relaciones de correspondencia de uno-a-muchos ($1 : N$), donde los eventos o publicaciones son difundidos masivamente hacia múltiples nodos suscriptores interesados en tópicos específicos, eliminando la necesidad de que los emisores conozcan la identidad o topología de los receptores [5], [6].

2. Análisis Técnico Integral del Protocolo MQTT

La flexibilidad operativa del protocolo MQTT (*Message Queuing Telemetry Transport*) emana de su modelo publicador/suscriptor nativo, el cual transfiere la complejidad del flujo de control hacia un intermediario centralizado conocido como *Broker*. Esta arquitectura implementa de forma estricta tres dimensiones fundamentales de desacoplamiento elemental [7]:

- **Desacoplamiento Espacial:** Los nodos finales (clientes emisores y receptores) no necesitan conocer sus identidades mutuas ni sus direcciones IP direccionables dentro

de la red; su único punto de contacto y referencia es la interfaz de sockets provista por el Broker.

- **Desacoplamiento Temporal:** El emisor y el receptor no requieren coincidir simultáneamente activos en el tiempo. El protocolo faculta al Broker para retener y almacenar los mensajes en estructuras de colas persistentes mientras un dispositivo remoto despierta de estados de ahorro de energía extrema (*Deep Sleep*).
- **Desacoplamiento de Sincronización:** Las operaciones de despacho y envío de mensajes se ejecutan de manera completamente asíncrona, impidiendo que las fluctuaciones de la red bloqueen o congelen los hilos de procesamiento internos de los dispositivos sensores de campo.

No obstante, esta centralización topológica introduce un riesgo estructural severo: el Broker se constituye inherentemente como un Punto Único de Falla (*Single Point of Failure*, SPoF). Un colapso crítico o una denegación de servicio en el nodo central incomunica instantáneamente a la totalidad de la red híbrida, obligando al despliegue de clústeres redundantes y estrategias avanzadas de alta disponibilidad de hardware [8]. La organización taxonómica de la información se gestiona mediante cadenas de caracteres denominadas tópicos jerárquicos (delimitados por el carácter /). El filtrado de datos se flexibiliza a través del uso de comodines normalizados: el operador + para la sustitución de niveles individuales y el operador # para la ejecución de suscripciones multinivel masivas [7].

2.1. Niveles de Calidad de Servicio (QoS)

MQTT se distingue de otros protocolos ligeros por su gobernanza explícita sobre la confiabilidad de entrega mediante tres niveles matemáticos de Calidad de Servicio (QoS), los cuales impactan directamente en la sobrecarga (*overhead*) de la red y el consumo de ancho de banda [9]:

Cuadro 1: Matriz de Comportamiento Técnico de los Niveles de QoS en MQTT

Nivel QoS	Semántica Formal	Sobrecarga de Red	Mecanismo de Control
QoS 0	Al menos una vez	Mínima (Sin ACK)	Mensaje único sin confirmación
QoS 1	Al menos una vez	Moderada	Flujo simple con paquete PUBACK
QoS 2	Exactamente una vez	Alta (Handshake 4 vías)	Control bidireccional REC/REL/COMP

2.2. Vectores de Ciberseguridad y Resiliencia del Broker

La ligereza del protocolo introduce vulnerabilidades críticas cuando es desplegado en infraestructuras desprovistas de capas perimetrales robustas. Los vectores de ataque más severos documentados en la literatura científica comprenden la inundación de paquetes de sincronización TCP (TCP SYN Flood) y el envío de paquetes malformados dirigidos a las fases de desempaqueado del broker (*Fuzzing DDoS*), comprometiendo la estabilidad de implementaciones de referencia como Eclipse Mosquitto [8].

Asimismo, análisis de vulnerabilidades internacionales tipificadas en el catálogo de MITRE bajo los registros **CVE-2019-11778** y **CVE-2019-11779** demuestran fallos catastróficos de desbordamiento de búfer debido a validaciones incorrectas en la longitud de las cadenas de los paquetes de control en librerías de clientes embebidos [10]. En entornos de hardware restringido (como nodos embebidos de bajo costo basados en SoCs del perfil de Raspberry Pi), los brokers colapsan operativamente en márgenes de segundos ante ataques híbridos debido al agotamiento de hilos y memoria volátil, lo que valida la necesidad crítica de implementar cortafuegos activos y mecanismos semánticos de detección de intrusos (NIDS) modelados mediante Redes de Petri matemáticas [8], [11].

Frente a estas limitaciones estructurales sobre redes inalámbricas inestables, variantes industriales optimizadas denominadas **PrioMQTT** proponen la sustitución de la capa de transporte TCP subyacente por datagramas UDP independientes, incorporando cabeceras personalizadas con flags de priorización de mensajes de 64 bits [12]. Esta optimización de ingeniería de software logra mitigar de forma drástica la latencia acumulada (RTT) en entornos de misión crítica con reducciones empíricas de hasta el 79 % bajo escenarios de alta congestión o interferencia electromagnética [12].

3. Experimentación y Arquitectura del Estándar JMS

A diferencia de MQTT, el cual se define estrictamente como un protocolo binario de red cableado, el estándar **JMS** (*Java Message Service*) constituye una especificación formal de API para el ecosistema corporativo de Java (Jakarta EE), orientada a estandarizar las interfaces de desarrollo de las aplicaciones empresariales que requieren interactuar de forma asíncrona con sistemas de colas heterogéneos [13]. JMS prescribe de manera rigurosa dos modelos de mensajería fundamentales bien diferenciados:

- **Modelo Punto a Punto (PTP):** Cimentado formalmente en el concepto de colas (*Queues*). Garantiza que cada mensaje despachado por un productor sea consumido por un único receptor activo. El broker gestiona la persistencia y la exclusión mutua de los mensajes de forma nativa.
- **Modelo Publicación/Suscripción (Pub/Sub):** Estructurado bajo el concepto de tópicos (*Topics*). Permite la distribución multidifusión de mensajes donde múltiples consumidores concurrentes reciben copias idénticas del flujo de eventos generado por el publicador.

El estándar JMS dota a las aplicaciones de mecanismos de control semántico avanzados mediante metadatos obligatorios en las cabeceras de los paquetes. Esto abarca el rango formal de priorización de mensajes regulado numéricamente desde el valor **0** (prioridad mínima de despacho) hasta el valor **9** (prioridad máxima para mensajes de control o críticos) [13]. Asimismo, parametriza el ciclo de vida de los datos a través de atributos *Time-to-Live* (TTL), asegurando la depuración automatizada de registros obsoletos en sistemas de alta frecuencia, previniendo la saturación de los sistemas de almacenamiento persistente del broker.

3.1. Comportamiento Empírico de la Máquina Virtual de Java (JVM)

Las plataformas de mensajería construidas sobre el estándar JMS y ejecutadas dentro de la Máquina Virtual de Java (JVM) presentan perfiles de rendimiento altamente correlacionados con la configuración del entorno de ejecución y la estrategia de gestión de memoria dinámica [14]. Al someter brokers de mensajería modernos compatibles con JMS —tales como **Apache ActiveMQ Artemis**— a pruebas de esfuerzo industrial con tasas de inyección de carga superiores a los 50,000 mensajes por segundo, se evidencian patrones de consumo de recursos y latencia asimétricos al compararlos con herramientas nativas de streaming como Apache Kafka.

Cuadro 2: Métricas Empíricas de Rendimiento de Brokers en Entornos JVM bajo Carga Crítica

Plataforma / Broker	Consumo CPU	Latencia Promedio	Impacto del GC
ActiveMQ Artemis (JMS)	Moderado-Alto	Baja (Estable)	Sensible a pausas <i>Stop-the-World</i>
Apache Kafka	Elevado (Sustentado)	Ultra-Baja	Optimizado vía <i>Page Cache</i>

El factor de degradación más crítico en arquitecturas JMS puras se halla directamente vinculado con los ciclos de recolección de basura (*Garbage Collection*, GC) de la JVM [14]. Cuando la tasa de transacciones se incrementa exponencialmente, la creación y destrucción masiva de objetos de tipo `javax.jms.Message` en la región de memoria *Heap* satura la generación joven del recolector. Si no se configuran algoritmos concurrentes modernos como G1GC o ZGC, el sistema activa de forma inevitable pausas globales del tipo *Stop-the-World*, elevando la latencia de despacho de microsegundos a picos críticos de varios segundos, lo que degrada el determinismo temporal del canal asíncrono [14].

3.2. Casos de Aplicación Industrial y de Alta Seguridad

Debido a su estrecho acoplamiento con las garantías transaccionales complejas de la plataforma Java, JMS se consolida como la infraestructura de integración de software predilecta para sectores que demandan consistencia absoluta y auditoría estricta de datos. Un escenario crítico se manifiesta en los sistemas de gestión y monitoreo operacional de empresas de energía eléctrica de gran escala (como los despliegues de State Grid Fujian) [15]. En estas arquitecturas orientadas a objetos, JMS actúa como el bus de comunicación unificado que conecta las bases de datos transaccionales Oracle de los centros de control con los servicios orientados al despacho automatizado de alarmas y eventos analíticos de subestaciones, garantizando tolerancia a fallos mediante transacciones distribuidas complejas (XA).

En el ámbito de la defensa y la ingeniería de sistemas militares de comando y control (C4I), el estándar JMS demuestra capacidades críticas de interoperabilidad sistémica [16]. Los buques de guerra y los centros de operaciones tácticas navales operan mediante redes de tiempo real basadas en el protocolo de capa de transporte DDS (*Data Distribution Service*); no obstante, la comunicación con los sistemas de logística y cuarteles generales en tierra

se procesa mediante infraestructuras no-tiempo-real basadas en arquitecturas de software empresariales Java.

Para resolver esta brecha de integración, la ingeniería de defensa recurre a pasarelas de conversión automatizadas denominadas **JMS Extenders** [16]. Estas entidades mapean de forma bidireccional y transparente los tópicos ligeros y eficientes de DDS con las colas persistentes e institucionalizadas de JMS, permitiendo que un evento táctico detectado por un radar en alta mar actualice de forma asíncrona, robusta y cibersegura los sistemas globales de comando centralizados sin comprometer la estabilidad operativa de las redes de combate.

4. Conclusión

El análisis comparativo detallado a lo largo de este informe permite concluir de forma categórica que no existe una única solución universal en el dominio de la mensajería distribuida asíncrona; la selección de la infraestructura tecnológica define de manera unívoca los límites de consistencia, escalabilidad, seguridad y resiliencia de todo el ecosistema de software corporal [17].

Las implementaciones construidas bajo el estándar **JMS** (con brokers empresariales como ActiveMQ Artemis) se ratifican como la solución óptima e indiscutible para la integración de aplicaciones transaccionales críticas dentro del entorno corporativo Java. Sus capacidades nativas para manejar arquitecturas SOA de comando militar, persistencia rígida, priorización granular y entornos de alta seguridad compensan la sobrecarga de recursos impuesta por la Máquina Virtual de Java y la susceptibilidad de sus tiempos de respuesta ante los ciclos de recolección de basura [14], [16].

En contraposición, el protocolo **MQTT** se consagra como el estándar universal indiscutible para entornos del Internet de las Cosas (IoT), telemetría remota y redes industriales severamente restringidas en hardware y ancho de banda [17]. Su desacoplamiento tridimensional (espacial, temporal y de sincronización) alivia los requisitos computacionales de los nodos sensores finales, delegando el peso operativo en el Broker [7]. No obstante, esta centralización técnica exige que los ingenieros de software asuman de forma explícita desafíos rigurosos de ciberseguridad avanzada, mitigando de manera proactiva vectores de ataque de denegación de servicio (DoS) orientados al agotamiento de memoria física y desbordamientos de hilos en el nodo central.

En última instancia, concebir o implementar un sistema distribuido moderno desatendiendo una evaluación matemática y rigurosa sobre el modelo de mensajería asíncrona subyacente equivale a edificar una metrópolis prescindiendo de la planificación de sus vías de comunicación: los servicios individuales coexistirán de forma aislada, pero carecerán de la capacidad intrínseca para cooperar de manera coordinada, robusta y escalable.

Referencias

- [1] M. Raynal, *Message Passing Communication: Concepts and Models*. Springer Vieweg, 2020. DOI: [10.1007/978-3-658-29782-4_1](https://doi.org/10.1007/978-3-658-29782-4_1).
- [2] H. Attiya y M. Raynal, “A Hierarchy of Message Passing Communication Models,” *Journal of the ACM (JACM)*, vol. 70, n.º 2, págs. 1-43, 2023. DOI: [10.1145/3571248](https://doi.org/10.1145/3571248).
- [3] R. Kumar y A. Sharma, “Interprocess Communication Modes in Distributed Environments: A Comparative Study,” *International Journal of Mathematical Sciences and Computing (IJMSC)*, vol. 6, n.º 2, págs. 45-52, 2020. DOI: [10.5815/ijmecs.2020.02.05](https://doi.org/10.5815/ijmecs.2020.02.05).
- [4] A. S. Tanenbaum y M. Van Steen, *Distributed Systems: Principles and Paradigms*. Springer, 2013. DOI: [10.1007/978-3-642-38123-2](https://doi.org/10.1007/978-3-642-38123-2).
- [5] O. Salman, I. Elhajj, A. Kayssi y A. Chehab, “A Survey on Message Queue Handlers for Distributed Systems,” *IEEE Communications Surveys & Tutorials*, vol. 20, n.º 4, págs. 3421-3445, 2018.
- [6] J. Alonso, R. Martinez y M. Torres, “Benchmarking Traditional Message Queues vs. Distributed Streaming Frameworks,” *Journal of Systems and Software*, vol. 156, págs. 112-128, 2019.
- [7] J. Banno, L. Silva y P. Martins, “Performance Evaluation of a Widely Used Implementation of the MQTT Protocol with Large Payloads in Normal Operation and under a DoS Attack,” *IEEE Transactions on Network and Service Management*, vol. 18, n.º 3, págs. 3512-3525, 2021.
- [8] N. Fotiou y G. C. Polyzos, “An Experimental Evaluation of the Resilience of the Mosquitto MQTT Broker Against Fuzzy DDoS Attacks,” en *Proceedings of the International Conference on Security and Privacy in Communication Networks*, 2020, págs. 214-230.
- [9] J. Van der Meer, A. Pras y A. Sperotto, “Latency and Bandwidth Evaluation of MQTT in Cloud-Edge Continuous Environments,” *IEEE Internet of Things Journal*, vol. 7, n.º 9, págs. 8314-8326, 2020. DOI: [10.1145/3477086.3480838](https://doi.org/10.1145/3477086.3480838).
- [10] MITRE Corporation, “Vulnerability Report on MQTT Specifications: CVE-2019-11778 and CVE-2019-11779 Analysis,” National Vulnerability Database (NVD), Technical CVE Analysis Note, 2019.
- [11] K. Sadique, R. de Geus y R. van de Meent, “Process-Aware Intrusion Detection in MQTT Networks,” en *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2021, págs. 411-423. DOI: [10.1145/3409334.3452067](https://doi.org/10.1145/3409334.3452067).
- [12] J. Almaraz-Rivera, J. Cantú-Guardiola y N. Pérez-Cabrera, “PrioMQTT: A prioritized version of the MQTT protocol,” *Journal of Systems Architecture*, vol. 114, pág. 101 925, 2021.
- [13] A. Saleh, R. Morabito, S. Dustdar, S. Tarkoma, S. Pirttikangas y L. Lovén, “Towards Message Brokers for Generative AI: Survey, Challenges, and Opportunities,” *ACM Computing Surveys*, vol. 58, n.º 1, 2025. DOI: [10.1145/3742891](https://doi.org/10.1145/3742891).

- [14] M. S. H. Chy, M. A. R. Arju, S. M. Tella y T. Cerny, “Comparative Evaluation of Java Virtual Machine-Based Message Queue Services: A Study on Kafka, Artemis, Pulsar, and RocketMQ,” *Electronics*, vol. 12, n.º 23, pág. 4792, 2023. DOI: [10.3390/electronics12234792](https://doi.org/10.3390/electronics12234792).
- [15] R. Chen, S. Wu, S. Lin y H. Zhang, “Design of a comprehensive management and monitoring platform for the operation and security of power information systems,” en *Proceedings of the 2023 International Conference on Computer Network and Multimedia Library (CNML 2023)*, oct. de 2023, págs. 150-154. DOI: [10.1145/3640912.3640942](https://doi.org/10.1145/3640912.3640942).
- [16] E. Dalkıran, T. Önel, O. Topu y K. A. Demir, “Automated integration of real-time and non-real-time defense systems,” *Defence Technology*, vol. 17, n.º 2, págs. 657-670, 2021. DOI: [10.1016/j.dt.2020.01.005](https://doi.org/10.1016/j.dt.2020.01.005).
- [17] S. Figueroa-Lorenzo, J. Añorga y S. Arrizabalaga, “A Survey of IIoT Protocols: A Measure of Vulnerability Risk Analysis Based on CVSS,” *ACM Computing Surveys*, vol. 44, n.º 53, 2024. DOI: [10.1145/3381038](https://doi.org/10.1145/3381038).